Hybrid Post-Quantum Authentication from OPAQUE and ML-KEM-768: Construction, Formal Analysis, and a Reproducible Implementation

Oleksandr Melnychenko[a]

[a]*Khmelnytskyi National University, Khmelnytskyi, Ukraine*

Abstract

Password authentication remains embedded in contemporary information systems, including systems that already use modern transport security. Password-authenticated key exchange, including OPAQUE, still relies on classical hardness assumptions tied to Diffie–Hellman problems and elliptic-curve cryptography. In settings where authentication transcripts may remain sensitive for years, scalable quantum attacks and the store-now-decrypt-later threat make hybrid post-quantum designs a practical migration question.

This paper specifies Hybrid PQ-OPAQUE, an OPAQUE-based construction in which ML-KEM-768 (FIPS 203) is incorporated into the authenticated 4DH exchange path without adding a protocol message. The session key is derived by HKDF-Extract from four Ristretto255 Diffie–Hellman values and the ML-KEM-768 shared secret under a transcript-dependent salt. The specification fixes the wire format, account-context binding through the account identifier acct, deterministic derivation of the OPRF key from the root secret σ_{oprf}, and transcript-level incorporation of the post-quantum components.

The analysis combines ProVerif 2.05, a surrogate 4DH model in Tamarin Prover 1.10.0, and a reference implementation in Rust validated by deterministic, integration, and property-oriented tests. The formal results are interpreted within explicit boundaries: they do not constitute a machine-checked proof of the implementation, and the offline-resistance claim is limited to compromise of the server database without compromise of σ_{oprf}.

On Apple M1 Pro, end-to-end authentication takes 481.43 ms, a full ML-KEM-768 round costs 122.01 μs, and Argon2id is the main latency component. The post-quantum layer adds 159.3 μs to the key-exchange path

without the KSF and increases the handshake size to 2715 bytes while preserving the three-message interaction profile. Relative to classical OPAQUE, the construction adds limited computation for honest parties while moving most overhead to message expansion.

Keywords: OPAQUE, PAKE, ML-KEM-768, post-quantum cryptography, hybrid key exchange, Ristretto255, HKDF, Tamarin, ProVerif, Rust

---

1. Introduction

1.1. Motivation

Password authentication remains widely deployed in contemporary distributed systems. Survey evidence indicates that more than 80% of authentication procedures in web applications and network services still rely on passwords [1, 2]. Password-authenticated key-exchange protocols (PAKEs) allow two parties sharing a low-entropy secret to establish a protected channel without revealing that secret to an adversary [3]. Among augmented PAKE constructions, OPAQUE [4] is a well studied scheme: under compromise of the authentication database alone, the server stores a cryptographic record rather than a password equivalent [5]. In deployed systems, this property depends on a clear trust boundary, because compromise of the global secret from which OPRF keys are derived is distinct from compromise of a passive record store.

This reliance on classical hardness assumptions is a concern in the presence of quantum adversaries. Shor's algorithm [6] applies to cryptosystems based on the discrete logarithm problem, including elliptic-curve key exchange and PAKE constructions built on related assumptions [7]. Earlier estimates suggested that breaking Ristretto255/Curve25519 would require a cryptographically relevant quantum computer with approximately 2,330 logical qubits [8]. More recent work reports two resource profiles for Shor's algorithm against 256-bit ECDLP over the secp256k1 curve: no more than 1200 logical qubits with 90 million Toffoli operations, or no more than 1450 logical qubits with 70 million Toffoli operations [9]. These figures are not parameter-specific estimates for Ristretto255. They are used here only as a contemporary calibration point for 256-bit elliptic-curve migration risk. The store-now-decrypt-later strategy [10], in which an adversary archives encrypted traffic for future decryption, adds to that motivation for systems protecting long-lived confidential data.

In August 2024, NIST published the first post-quantum cryptography standards, including FIPS 203 for ML-KEM [11] and FIPS 204 for ML-DSA [12]. Hybrid constructions that combine classical and post-quantum components are being introduced in TLS 1.3 [13], Signal (PQXDH) [14], and WireGuard [15]. Password authentication has not reached the same level of standardization. Classical OPAQUE in its current IETF specification [5] contains no mechanism aimed at quantum attacks, and other established PAKE protocols such as SRP and SPAKE2 have the same limitation. Recent work on post-quantum and hybrid PAKE has produced compact KEM-based constructions [16], UC/QROM-secure schemes [17], and formal studies of hybrid composition [18, 19]. What remains less developed is the combination of an augmented-PAKE design with explicit message syntax, scoped formal claims, measurements, and implementation-level evidence.

1.2. Research Scope and Contributions

This paper studies a narrower engineering question: whether OPAQUE can be reinforced with ML-KEM-768 while preserving its three-message authentication flow. The construction combines four Ristretto255 Diffie–Hellman values with an ML-KEM-768 shared secret, binds an explicit account context represented by the account identifier acct and wire-format versioning into the transcript, and evaluates the result using symbolic analysis and an executable reference implementation in Rust. The contribution is not a general theory of hybrid PAKE composition; it is an implementation-level augmented-PAKE design with explicit syntax, stated assumptions, and a deliberately limited claim set.

The analysis is organized around four questions: whether ML-KEM-768 can be incorporated without increasing the number of protocol messages; how the classical and post-quantum key materials should be combined to preserve transcript binding; what evidence can support implementation-level claims without overstating them; and whether the added post-quantum cost matters when Argon2id dominates end-to-end latency. These questions are addressed by keeping the protocol description, symbolic models, tests, and benchmarks aligned to one implementation-oriented specification. Relative to classical OPAQUE, the resulting design preserves the interaction profile, adds moderate computational overhead outside the password-hardening step, and moves most of the new cost to message expansion in KE1 and KE2.

1.3. Research Contributions

1. **A concrete hybrid 4DH + ML-KEM-768 construction.** The paper specifies a Hybrid PQ-OPAQUE variant in which ML-KEM-768 is part of the authenticated key exchange rather than an optional extension.
2. **A transcript-bound key-material combiner.** The pre-key is formed as in Eq. (16), with a context-dependent salt containing a domain-separated hash of the extended transcript τ.
3. **An implementation-level protocol specification.** Beyond the post-quantum elements themselves, the transcript includes an explicit account-context hash, while the wire format fixes a one-byte version prefix and exact message sizes.
4. **Layered formal and implementation support.** Reduction sketches, two ProVerif models, a Tamarin model with a surrogate 4DH representation, and Rust tests are aligned to the same construction and the same threat-model boundaries.
5. **Executable artifact and measurements.** The artifact contains implementation code, tests, and modules for micro-benchmarking, protocol-level benchmarking, and throughput assessment.
6. **Measured empirical overheads.** The post-quantum layer increases key-exchange CPU cost by 159.3 μs without the KSF, while Argon2id remains the main contributor to end-to-end latency.

1.4. Terminological Conventions

To avoid terminological drift, the remainder of the paper adheres to the vocabulary fixed in Table 1. Subsequent uses of these terms should be read in the precise senses stated there.

In addition, the formal exposition uses acct for the account identifier and σ_{oprf} for the root secret from which server-side OPRF keys are derived. Literal ASCII domain-separation strings are replaced below with the symbols L_{oprf}, L_{seed}, L_{key}, L_{ksf}, L_{salt}, L_{ctx}, L_{τ}, and L_{comb}; for the purposes of formal analysis, their stability and mutual distinctness matter, whereas the concrete textual representation of the implementation string does not.

2. Related Work and Existing Solutions

2.1. Password-Authenticated Key-Exchange Protocols

Password-authenticated key exchange (PAKE) refers to a class of protocols that enables two parties sharing a low-entropy secret (a password)

Table 1: Terminological conventions adopted in this paper

Term	Fixed meaning
Wire format	The complete byte-level representation of a message on the channel, including the version prefix and protocol service fields
Payload	The message body excluding the version prefix and any external wrappers
Account identifier (acct)	A stable byte string identifying the user's account; in this construction it is cryptographic context, not only a database lookup key
Augmented PAKE (aPAKE)	A class of password-authenticated key-exchange protocols in which the server stores a verifier rather than the password itself or its direct equivalent
Key-derivation scheme	The sequence of extraction and expansion procedures that produces session and auxiliary keys
Research artifact	Source code, formal models, tests, and benchmarks needed for independent reproduction of the reported results

to establish an authenticated session key resistant to offline dictionary attacks [20]. PAKE protocols are commonly divided into two categories:

- *Balanced PAKE*, in which both parties store equivalent password-derived representations. Representative examples are EKE [21], SPAKE2 [22], and CPace/AuCPace [23].
- *Augmented PAKE (aPAKE)*, in which the server stores only a verifier rather than the password or its direct equivalent. Representative examples are SRP [24], AuCPace [23], and OPAQUE [4].

In Table 2, the OPAQUE entry regarding server storage compromise should be read in the narrow but standard sense of compromise of the authentication-record database alone. It does not cover compromise of a global server secret from which OPRF keys may be derived.

2.2. OPAQUE: A Formal Overview

OPAQUE [4] consists of a registration phase and an authentication phase, both built around an oblivious pseudorandom function (OPRF). Standard IETF OPAQUE uses triple Diffie–Hellman (3DH): $dh_1 = sk_S \cdot PK_C$ (mutual

Table 2: Comparative characteristics of selected PAKE protocols

Property	EKE	SRP	SPAKE2	OPAQUE
Type	Balanced	Augmented	Balanced	Augmented
Non-equivalent server record	No	Partial	No	Yes
Pre-computation resistance	No	No	No	Yes
Forward secrecy	Yes	Yes	Yes	Yes
Resistance to server-record compromise	No	Partial	No	In the passive-database model
Formal security proof	IC	None	ROM	UC
Quantum resistance	No	No	No	No

authentication), $dh_2 = sk_S \cdot E_C$, $dh_3 = e_S \cdot PK_C$ (forward secrecy). The construction analyzed here adds a fourth product, $dh_4 = e_S \cdot E_C$. This term binds the two ephemeral components within a single transcript and, at the level of 4DH intuition, is intended to complicate UKS-like substitutions; here it is treated as a design rationale rather than as an independently proved property. Baseline OPAQUE has been proved in the Universal Composability (UC) model [25].

Oblivious pseudorandom function (OPRF). The implementation uses hashing into the Ristretto255 group [26]:

$$\alpha = r \cdot H'(\mathrm{ctx} \| x), \quad \beta = k \cdot \alpha, \quad F_k(x) = r^{-1} \cdot \beta \tag{1}$$

where $r \xleftarrow{\$} \mathbb{Z}_q^*$ is a random client-side blinding scalar and k is the server's secret key.

2.3. Post-Quantum Cryptography and the NIST 2024 Standards

ML-KEM-768 is selected at NIST security level 3 (roughly aligned with AES-192). Its parameters are $|pk| = 1184$ bytes, $|ct| = 1088$ bytes, and $|ss| = 32$ bytes under IND-CCA2 security. The abstraction is

$$\mathrm{KeyGen}() \to (pk, sk); \quad \mathrm{Encaps}(pk) \to (ct, ss); \quad \mathrm{Decaps}(sk, ct) \to ss. \tag{2}$$

2.4. Rationale for Selecting ML-KEM-768

The selection of ML-KEM-768 follows from the deployment constraints of interactive password authentication. ML-KEM-512 offers smaller messages and lower overhead, but it targets NIST level 1. ML-KEM-1024 provides a higher nominal security level, but it also increases message size and transport cost. For interactive password authentication, where the protocol already

includes a comparatively expensive KSF, the relevant design criterion is the balance among cryptographic strength, message size, and deployability.

ML-KEM-768 provides a middle operating point in that trade-off space. Within the ML-KEM family it aligns NIST level 3 with the long-term protection of account credentials and session secrets while avoiding the larger transport burden imposed by ML-KEM-1024. Although pk_{kem} and ct_{kem} enlarge messages relative to classical OPAQUE, they do not render the protocol unsuitable for interactive use.

2.5. Design Alternatives Considered

Several hybridization strategies are possible, and they lead to different operational trade-offs. A purely transport-layer migration, for example, can use a post-quantum or hybrid TLS key exchange while leaving the application authentication protocol unchanged. That path is attractive for deployment, but it does not make the password-authentication transcript itself hybrid. Conversely, replacing the OPRF with a post-quantum OPRF would address the password-randomization layer more directly, but efficient and standardized post-quantum OPRFs are less mature than ML-KEM-based key establishment. A third option is to store long-lived post-quantum material in the registration record. This would increase migration pressure on account state and would enlarge the server-side record that operators must protect.

The construction studied here therefore adopts a narrower design: keep the OPRF and envelope structure close to OPAQUE, add ML-KEM-768 only in the ephemeral authentication path, and bind the resulting public KEM values into the same transcript that authenticates the classical exchange. This choice does not address every post-quantum PAKE problem, but it gives a concrete migration point whose wire format, implementation cost, and trust boundary can be measured.

2.6. Recent Approaches to Post-Quantum PAKE (2023–2026)

Between 2023 and 2026, several research lines became relevant to post-quantum PAKE and hybrid aPAKE design.

KEM-based schemes indicate that compact lattice-based PAKE is feasible. A representative example is CHIC [16], which combines a compact construction with a UC-security argument and implementation-level evaluation. In parallel, UC- and QROM-oriented results such as [17] strengthen compositional guarantees for the post-quantum setting. These results are

Table 3: Hybridization strategies and their implications for this work

Strategy	Benefit	Limitation for the present objective
Hybrid transport only	Minimal application changes when the surrounding channel already supports post-quantum key exchange	The PAKE transcript and application-level authentication state remain classical.
Post-quantum OPRF replacement	Targets the password-randomization layer directly	Efficient post-quantum OPRF choices are less standardized and would change the registration and envelope design more deeply.
Persistent PQ material in the record	Can attach post-quantum state to the account itself	Enlarges long-lived account records and complicates staged migration.
Separate PAKE and KEM composition	Keeps the classical PAKE largely unchanged	Requires a separate composition argument and careful binding to avoid treating two independent handshakes as one authenticated exchange.
Ephemeral ML-KEM in the OPAQUE transcript	Preserves three messages, keeps PQ material out of the registration record, and exposes concrete wire-format costs	Leaves the OPRF layer classical and requires a carefully scoped hybrid-combiner interpretation.

important, but they do not directly resolve the migration problem for deployed OPAQUE-like systems, where asymmetric client–server roles, wire-format stability, and compatibility with existing authentication workflows are central.

A second line of work considers replacement of classical OPRF components and the compilation of KEM primitives into PAKE. Lattice-based post-quantum OPRF constructions remain more communication-intensive than elliptic-curve instantiations [27], and generalized KEM-to-PAKE approaches require careful treatment of ideal-cipher assumptions, hashing into public-key spaces, and low-entropy secrets [16]. By contrast, isogeny-based approaches have lost practical relevance after the Castryck–Decru attacks on SIDH-derived constructions [28].

Recent work also treats hybridization itself as a distinct design problem. In particular, [18] studies PAKE combiners with efficient post-quantum instantiations, whereas [19] analyzes hybrid PAKE in the UC framework via parallel and sequential composition. An IETF draft on hybrid post-quantum augmented PAKE appeared in 2025 within the CFRG context [29]. The present work follows this line by studying a concrete hybrid extension of OPAQUE with explicit message syntax, bounded formal claims, and a reproducible implementation.

3. Problem Statement

3.1. Problem Formulation and Threat Model

Let $\mathcal{C}$ (client/initiator) and $\mathcal{S}$ (server/responder) be the two parties of the protocol, where $\mathcal{C}$ holds the password pwd and $\mathcal{S}$ stores the registration record rec. Let $\mathbb{G}$ denote the prime-order Ristretto255 group of order q with generator g.

Assets to be protected:

- the password pwd; the static keys sk_C, sk_S; the ephemeral keys $e_C, e_S, sk_{\text{kem}}$;
- the registration record rec = (envelope, pk_C);
- the session key K_s, the master key K_m, and the shared secret ss_{kem}.

Adversarial model. Three adversarial classes are considered:

- *Classical adversary* $\mathcal{A}_C$: a probabilistic polynomial-time active man-in-the-middle capable of compromising server-side storage;
- *Quantum adversary* $\mathcal{A}_Q$: an adversary modeled as capable of solving DLP/ECDLP via Shor's algorithm and pursuing a "harvest now, decrypt later" strategy;
- *Composite adversary* $\mathcal{A}_{CQ}$: an active man-in-the-middle equipped with a quantum computational oracle.

Threat-model boundary. The adversary controls the network and may record, replay, delay, drop, and modify protocol messages. The database-compromise scenario gives the adversary the registration record and associated public metadata, but not the operational secret σ_{oprf} from which account-specific OPRF keys are derived. The model also permits post-session compromise of long-term static protocol keys when discussing forward secrecy. It does not include full endpoint compromise during an active session, malicious random number generation, password-reset workflows, denial-of-service economics, or microarchitectural side channels. These excluded cases are operationally important, but they require different mechanisms from the protocol-level construction studied here.

Protocol requirements:

Table 4: Threat-model boundary used for the main security claims

Scenario	Included in the claim	Boundary
Active network attack	Full Dolev–Yao control of messages and transcript manipulation	Does not model traffic analysis or availability attacks.
Database disclosure	Registration record, stored public keys, and protocol metadata may be exposed	Offline resistance assumes σ_{oprf} remains outside the compromised store.
Post-session key exposure	Long-term static keys may be compromised after completion	Session-local ephemerals and ML-KEM decapsulation state are assumed destroyed.
Quantum attack on classical DH	Classical DH terms may be treated as recoverable in the long-term migration argument	The post-quantum layer is then relied upon under the ML-KEM assumption.
Implementation behavior	Serialization, state lifecycle, version handling, and external-adapter errors are checked by tests	Tests do not establish absence of side channels or memory-safety bugs in external crates.

R1 (OPAQUE-derived goals): password secrecy, forward secrecy, mutual authentication, and resistance to offline guessing;

R2 (Post-quantum reinforcement): inclusion of a KEM-based contribution against "harvest now, decrypt later" exposure;

R3 (AND interpretation): the key-material combiner should be compatible with an AND-style interpretation in the adopted model, so that explicit recovery of the final key cannot be reduced to weakening only one layer;

R4 (Interaction preservation): preservation of three protocol messages and 1.5 network round trips;

R5 (Implementability): a reproducible reference implementation in Rust suitable for further analysis and engineering.

3.2. Formal Definitions

Definition 3.1 (KEM)**.** (KeyGen, Encaps, Decaps)*:* $\mathrm{KeyGen}() \to (pk, sk)$*;* $\mathrm{Encaps}(pk) \to (ct, ss)$*;* $\mathrm{Decaps}(sk, ct) \to ss$*. IND-CCA2 security means that no polynomial-time adversary equipped with a decapsulation oracle can distinguish the real ss from a random string.*

Definition 3.2 (Hybrid aPAKE in the sense of this paper)**.** *Protocol* Π *between* $\mathcal{C}(\mathrm{pwd})$ *and* $\mathcal{S}(\mathrm{rec})$*, in which*

$$\mathrm{IKM}_{\mathrm{cl}} = dh_1 \| dh_2 \| dh_3 \| dh_4, \qquad \mathrm{IKM}_{\mathrm{pq}} = ss_{\mathrm{kem}},$$

and the final pre-key is formed as

$$\mathrm{PRK} = \mathrm{HKDF\text{-}Extract}\Big(\mathrm{salt}_\tau,\ \mathrm{IKM}_{\mathrm{cl}} \| \mathrm{IKM}_{\mathrm{pq}}\Big).$$

The subsequent analysis examines the extent to which such a construction is compatible with an AND-style interpretation of security within the threat model considered here.

Definition 3.3 (Pseudorandomness of HKDF-Extract)**.** *HKDF-Extract based on HMAC-SHA-512 is modeled as an extractor: if* IKM *contains hidden computational entropy relative to the adversary, then the output PRK is computationally indistinguishable from uniform up to advantage at most* ε*.*

Notation: $\mathbb{G}$ denotes Ristretto255 with $|\mathbb{G}| = q$; lowercase letters denote scalars; uppercase letters denote curve points; $\|$ denotes concatenation; $H(\cdot)$ denotes SHA-512; and $\mathcal{K} = (K_s, K_m)$ denotes the session and master keys.

4. The Proposed Hybrid PQ-OPAQUE Protocol

4.1. Architecture and Overview

Hybrid PQ-OPAQUE is a hybrid post-quantum extension of OPAQUE [5]. The protocol preserves the defining property of augmented PAKE, namely that the server never learns the password, while adding a post-quantum component for the "harvest now, decrypt later" threat model. The protocol consists of two phases: *registration* (a one-time phase) and *authentication* (the three-message KE1–KE2–KE3 exchange). Throughout the sequel, the terms *wire format* and *payload* are used in the senses fixed in Table 1.

Table 5: Cryptographic primitives used in Hybrid PQ-OPAQUE

Component	Primitive	Implementation	Security level
OPRF group	Ristretto255	curve25519-dalek	128 bits
Hash function	SHA-512	sha2	256 bits
KDF (extract)	HKDF-Extract (HMAC-SHA-512)	hmac + sha2	256 bits
KDF (expand)	HKDF-Expand (HMAC-SHA-512)	hmac + sha2	256 bits
MAC	HMAC-SHA-512	hmac + sha2	256 bits
KSF	Argon2id (MODERATE)	argon2	Adaptive
AEAD	XSalsa20-Poly1305	crypto_secretbox	256 bits
KEM	ML-KEM-768 (FIPS 203)	ml-kem	NIST Level 3
Classical exchange	4DH (Ristretto255)	curve25519-dalek	128 bits
Constant-time comparison	Bytewise without early exit	subtle	—

Figure 1 shows that the post-quantum layer leaves finalization almost unchanged while increasing the transport budget of the first two messages, namely the phases that carry pk_{kem} and ct_{kem}.

Table 6: Authentication-message sizes in payload and wire format

Message	Classical payload	Hybrid payload	Hybrid wire format	Post-quantum overhead
KE1	88 bytes	1272 bytes	1273 bytes	+1184 (pk_{kem})
KE2	288 bytes	1376 bytes	1377 bytes	+1088 (ct_{kem})
KE3	64 bytes	64 bytes	65 bytes	0
Total	**440 bytes**	**2712 bytes**	**2715 bytes**	+2272 (+516%)

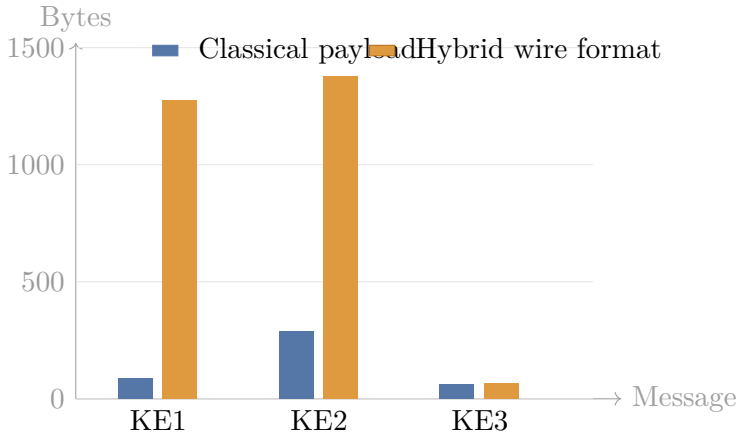

Figure 1: Comparison of authentication-message sizes: the additional wire-format overhead is concentrated in KE1 and KE2, whereas KE3 remains essentially unchanged.

4.2. Registration Phase

Step 1 ($\mathcal{C} \to \mathcal{S}$). Generation of the static pair (sk_C, PK_C) followed by OPRF blinding:

$$r \xleftarrow{\$} \mathbb{Z}_q^*, \qquad B = r \cdot H'\Big(L_{\text{oprf}} \| \texttt{0x00} \| \text{pwd}\Big) \tag{3}$$

RegistrationRequest $= B$ (32 bytes).

Step 2 ($\mathcal{S} \to \mathcal{C}$). Derivation of the OPRF key:

$$s_{\text{oprf}} = \text{HMAC}\Big(\sigma_{\text{oprf}}, L_{\text{seed}}\Big)[0..31] \tag{4}$$

$$k_{\text{oprf}} = \text{ScalarReduce}\Big(\text{HMAC}\Big(s_{\text{oprf}},\ L_{\text{key}} \| \mathsf{acct} \| \text{ctr}\Big)\Big) \tag{5}$$

where ctr is the smallest counter for which the resulting scalar is non-zero. The server computes $Z = k_{\text{oprf}} \cdot B$; the key k_{oprf} is then immediately destroyed. The response is $Z \| PK_S$ (64 bytes).

Step 3 ($\mathcal{C}$). OPRF unblinding:

$$U = r^{-1} \cdot Z.$$

OPRF output:

$$y_{\text{oprf}} = H\Big(L_{\text{oprf}} \| \texttt{0x01} \| \text{pwd} \| U\Big) \tag{6}$$

KSF input and salt:

$$x_{\text{ksf}} = H\Big(L_{\text{ksf}} \| y_{\text{oprf}} \| \text{pwd}\Big), \tag{7}$$

$$\text{salt}_{\text{ksf}} = H\Big(L_{\text{salt}} \| y_{\text{oprf}}\Big)[0..15]. \tag{8}$$

The randomized password is then computed with Argon2id [30] as

$$\text{rwd} = \text{Argon2id}\Big(x_{\text{ksf}},\ \text{salt}_{\text{ksf}};\ m = 256 \text{ MiB},\ t = 3,\ p = 1\Big) \tag{9}$$

The envelope is sealed using XSalsa20-Poly1305 and a key derived via HKDF; RegistrationRecord $=$ Envelope$\| PK_C$ (168 bytes of payload and 169 bytes in wire format, including the version prefix).

4.3. Binding of Account Identifier, Version, and Context

The protocol fixes additional context that remains implicit in many abstract descriptions of OPAQUE. Every wire-format message carries a one-byte version prefix $v = \texttt{0x01}$, which separates the payload from the full transmission format and enables rejection of unsupported versions. The protocol also uses an explicit account identifier acct, a stable byte string naming the account, treated not merely as a database lookup key but as cryptographic context.

Let

$$\eta = H\Big(L_{\mathrm{ctx}} \| \mathsf{acct}\Big) \tag{10}$$

Distinct domain-separation labels separate the baseline OPRF, KSF, and transcript contexts from the added account-binding and hybrid-combiner contexts. The version indices embedded in these labels are local domain identifiers rather than global version numbers of the overall scheme. The identifier acct influences the protocol at two points:

1. through derivation of an account-specific OPRF key, as in Eq. (5);
2. through inclusion of η, defined in Eq. (10), in the extended transcript (15), and hence in the hybrid combiner (16) and the derived keys (17).

This binding makes cross-account transplantation visible in transcript verification even when the password is identical, and it makes the role of account identity explicit in the cryptographic specification.

4.4. Authentication Phase and Hybrid Key Exchange

4.4.1. Generation of KE1 from Client to Server

The client first samples the OPAQUE authentication randomness and creates the ephemeral ML-KEM key pair advertised in the first message:

$$\begin{aligned} &e_C \xleftarrow{\$} \mathbb{Z}_q^*, && E_C = e_C \cdot g, \\ &n_C \xleftarrow{\$} \{0,1\}^{192}, && (pk_{\mathrm{kem}}, sk_{\mathrm{kem}}) \leftarrow \text{ML-KEM-768.KeyGen}(). \end{aligned} \tag{11}$$

The resulting first authentication message is

$$\begin{aligned} \mathrm{KE1} &= B \,\|\, E_C \,\|\, n_C \,\|\, pk_{\mathrm{kem}}, \\ |\mathrm{KE1}| &= 32 + 32 + 24 + 1184 = 1272 \text{ bytes}. \end{aligned} \tag{12}$$

With the one-byte protocol version prefix, the wire message $v \,\|\, \mathrm{KE1}$ occupies 1273 bytes.

4.4.2. Generation of KE2 from Server to Client

Four-way Diffie–Hellman (4DH):

$$dh_1 = sk_S \cdot PK_C, \quad dh_2 = sk_S \cdot E_C, \quad dh_3 = e_S \cdot PK_C, \quad dh_4 = e_S \cdot E_C. \tag{13}$$

ML-KEM-768 encapsulation:

$$(ct_{\text{kem}}, ss_{\text{kem}}) \leftarrow \text{ML-KEM-768.Encaps}(pk_{\text{kem}}). \tag{14}$$

Extended transcript (including post-quantum elements):

$$\begin{aligned} \tau &= E_C \| E_S \| n_C \| n_S \| PK_C \| PK_S \| \text{CredResp} \| pk_{\text{kem}} \| ct_{\text{kem}} \| \eta \\ \tau_H &= H\Big(L_\tau \| \tau\Big) \end{aligned} \tag{15}$$

Hybrid combiner:

$$\begin{aligned} \text{combined\_ikm} &= dh_1 \| dh_2 \| dh_3 \| dh_4 \| ss_{\text{kem}} \quad (128 + 32 = 160 \text{ bytes}) \\ \text{salt} &= L_{\text{comb}} \| \tau_H \\ \text{PRK} &= \text{HKDF-Extract}(\text{salt}, \text{combined\_ikm}) \end{aligned} \tag{16}$$

Key derivation. Let Λ_{s}, Λ_{m}, Λ_{macS}, and Λ_{macC} denote fixed labels for the session key, the master key, and the two confirmation keys:

$$\begin{aligned} K_s &= \text{HKDF-Expand}(\text{PRK}, \Lambda_{\text{s}}, 64) \\ K_m &= \text{HKDF-Expand}(\text{PRK}, \Lambda_{\text{m}}, 32) \\ K^S_{\text{mac}} &= \text{HKDF-Expand}(\text{PRK}, \Lambda_{\text{macS}}, 64) \\ K^C_{\text{mac}} &= \text{HKDF-Expand}(\text{PRK}, \Lambda_{\text{macC}}, 64) \end{aligned} \tag{17}$$

$$\text{KE2} = n_S \| E_S \| \text{CredResp} \| \tag{18}$$

$$\text{HMAC}(K^S_{\text{mac}}, \tau) \| ct_{\text{kem}} \quad (1376 \text{ bytes of payload}). \tag{19}$$

In wire format, the message is $v \| \text{KE2}$ and occupies 1377 bytes.

4.4.3. Generation of KE3 from Client to Server

1. OPRF finalization, opening of the envelope, and recovery of the server public key, client static key, and client public key;
2. Mirror 4DH: $dh_1 = sk_C \cdot PK_S$, $dh_2 = e_C \cdot PK_S$, $dh_3 = sk_C \cdot E_S$, $dh_4 = e_C \cdot E_S$ (additional binding of the ephemeral components at the level of 4DH intuition);

3. Decapsulation: ss_{kem} = ML-KEM-768.Decaps($sk_{\text{kem}}, ct_{\text{kem}}$); sk_{kem} is destroyed immediately;
4. Re-derivation of the same PRK; verification of HMAC(K^S_{mac}, τ) by means of a constant-time bytewise comparison without early exit;
5. KE3 = HMAC(K^C_{mac}, τ) (64 bytes of payload, 65 bytes including the version prefix).

4.4.4. Completion on the Server Side

KE3 is verified using a constant-time comparison routine. Upon success, the pair (K_s, K_m) becomes available to both parties, thereby confirming mutual authentication. Upon failure, all key material is destroyed immediately via secure zeroization.

4.5. Inclusion of Post-Quantum Elements in the Transcript

The inclusion of pk_{kem} and ct_{kem} in the transcript binds the post-quantum exchange to the classical one. Without this binding, a man-in-the-middle adversary could replace pk_{kem} with an attacker-controlled key at the ML-KEM layer without that substitution being reflected in the MAC input. This linkage is captured by the extended transcript (15) and the hybrid combiner (16). The additional inclusion of the account-context hash η makes account substitution visible to the transcript computation. The resulting transcript binds the classical and post-quantum layers to each other and to a concrete account and wire-format version.

5. Security Analysis

5.1. Formal Analysis of Security Properties

Offline guessing under a fixed trust boundary.. The relevant compromise case is deliberately narrow. The adversary is allowed to obtain the passive server-side storage rec and the observable transcripts, but not σ_{oprf}. Under that condition, the record does not give an efficient offline verifier for dictionary candidates except through a break of OPRF obliviousness or envelope security. If σ_{oprf} is compromised, the problem changes: the attacker can evaluate the account-specific OPRF path and the remaining work is a dictionary search with per-guess cost dominated by Argon2id. The claim is therefore a statement about a specific operational boundary, not about all possible server compromise cases.

Proof sketch. **Case 1:** σ_{oprf} not compromised. Without the account-specific OPRF key, the adversary cannot compute y_{oprf} via (6), and hence cannot reconstruct x_{ksf}, salt_{ksf}, or rwd via (9). The envelope record therefore does not provide an effective offline password-checking oracle. Replacing the OPRF output with a random value yields

$$|\Pr[S_1] - \Pr[S_0]| \leq \text{Adv}^{\text{obliv}}_{\text{OPRF}} \leq \text{Adv}^{\text{CDH}}_{\text{Ristretto255}}.$$

Envelope-hiding transition. Next replace the envelope with an encryption of a random string, reducing the change in success probability to IND-CPA security of XSalsa20-Poly1305:

$$|\Pr[S_2] - \Pr[S_1]| \leq \text{Adv}^{\text{IND-CPA}}_{\text{XSalsa20}}.$$

Consequence. In the absence of compromise of σ_{oprf}, the best available strategy collapses to online guessing, whence

$$\text{Adv}^{\text{dict}} \leq \text{Adv}^{\text{CDH}}_{\text{Ristretto255}} + \text{Adv}^{\text{IND-CPA}}_{\text{XSalsa20}} + \frac{q_{\text{on}}}{|\mathcal{D}|}.$$

Case 2: σ_{oprf} compromised. Then each dictionary guess pwd' can be checked offline by computing y'_{oprf}, then x'_{ksf}, $\text{salt}'_{\text{ksf}}$, and finally one attempt to open the envelope. The cost of a single guess is dominated, both asymptotically and empirically, by Argon2id, that is, by the procedure (9):

$$T_{\text{guess}} \approx T_{\text{Argon2id}}, \qquad C_{\text{offline}} \approx |\mathcal{D}| \cdot T_{\text{guess}}.$$

Claim 5.1 (Scoped forward secrecy). Compromise of the long-term keys sk_C and sk_S after session completion does not suffice to recover K_s, under the hardness of CDH and the IND-CCA security of ML-KEM-768.

Proof sketch. After compromise of both long-term keys, the terms containing a static secret can be reconstructed from the public transcript: $dh_1 = sk_S \cdot PK_C$, $dh_2 = sk_S \cdot E_C$, and $dh_3 = sk_C \cdot E_S$. The remaining classical term $dh_4 = e_S \cdot E_C = e_C \cdot E_S$ still requires recovery of a session ephemeral scalar or a break of CDH on the recorded ephemeral points. In parallel, ss_{kem} remains protected under the IND-CCA security of ML-KEM-768 because sk_{kem} is destroyed after decapsulation. Thus post-session exposure of long-term keys alone does not reconstruct the full input to the hybrid extractor:

$$\text{Adv}^{\text{FS}} \leq \min\left\{\text{Adv}^{\text{Ristretto255}}_{\text{CDH}}, \text{Adv}^{\text{IND-CCA}}_{\text{ML-KEM}}\right\} + \text{Adv}^{\text{ext}}_{\text{HKDF}}. \tag{20}$$

Claim 5.2 (Mutual authentication). Under the PRF model for HMAC-SHA-512, a successful MAC forgery is bounded by

$$\text{Adv}^{\text{auth}} \leq \text{Adv}^{\text{PRF}}_{\text{HMAC}} + q_s/2^{512}.$$

Interpretive claim on the AND property of the hybrid combiner.. Let

$$\mathrm{IKM}_{\mathrm{cl}} = dh_1 \| dh_2 \| dh_3 \| dh_4, \qquad \mathrm{IKM}_{\mathrm{pq}} = ss_{\mathrm{kem}}.$$

If, for a fixed transcript τ, at least one of the sources $\mathrm{IKM}_{\mathrm{cl}}$ or $\mathrm{IKM}_{\mathrm{pq}}$ remains computationally indistinguishable from uniform, then

$$\mathrm{PRK} = \text{HKDF-Extract}(\mathrm{salt}_\tau, \mathrm{IKM}_{\mathrm{cl}} \| \mathrm{IKM}_{\mathrm{pq}})$$

is treated as pseudorandom under the standard extractor interpretation of HKDF-Extract. This gives the intended AND-style reading of the key schedule: explicit recovery of the real session key from the transcript requires compromise of *both* sources. The statement is not a standalone computational proof of the combiner. It is an extractor-based interpretation that is used consistently in the subsequent symbolic model and implementation checks.
Proof sketch. Let S denote the source among $\{\mathrm{IKM}_{\mathrm{cl}}, \mathrm{IKM}_{\mathrm{pq}}\}$ that remains pseudorandom relative to the fixed transcript τ, and let X denote the other source. Then $\mathrm{IKM}_{\mathrm{cl}} \| \mathrm{IKM}_{\mathrm{pq}}$ may be written as $S \| X$. The standard extractor property of HMAC-based HKDF-Extract [31] suggests that if S is computationally indistinguishable from uniform, then PRK remains pseudorandom even for arbitrary X. This yields the upper bound: indistinguishability of the final key can fail only through a break of the surviving source or a break of the extractor itself:

$$\mathrm{Adv}^{\mathrm{hybrid}} \leq \min\left\{ \mathrm{Adv}^{\mathrm{Ristretto255}}_{\mathrm{CDH}}, \mathrm{Adv}^{\mathrm{IND\text{-}CCA}}_{\mathrm{ML\text{-}KEM}} \right\} + \mathrm{Adv}^{\mathrm{ext}}_{\mathrm{HKDF}}. \tag{21}$$

This bound explains the AND-style interpretation used in the sequel. If at least one of the two sources remains computationally hidden, and HKDF-Extract behaves as an extractor, then the derived PRK remains pseudorandom up to the additive extractor term. The interpretation is narrow: in this reduction sketch and in the symbolic analysis below, compromise of the final key is modeled as requiring simultaneous weakening of the classical and post-quantum layers.

5.2. Resistance to Known Attacks

Offline dictionary attacks. The protection claim is stated for *database compromise without compromise of* σ_{oprf}. Under this condition, three independent layers operate: (1) OPRF, because without the server-side key k_{oprf} the adversary cannot compute rwd; (2) Argon2id with MODERATE

parameters (256 MiB, 3 iterations), for which one guess costs approximately $T_{\text{guess}} \approx 0.5$ s on the benchmarking platform, so a sequential exhaustive dictionary search scales as $C_{\text{offline}} \approx |\mathcal{D}| \cdot T_{\text{guess}}$; and (3) XSalsa20-Poly1305, because verification of a candidate requires successful authenticated decryption. If σ_{oprf} is compromised, this claim does not automatically transfer to the new threat model and requires separate analysis.

Man-in-the-middle attack. The transcript τ contains the public protocol elements, including pk_{kem} and ct_{kem}. Modification of any field results in MAC mismatch.

Replay attack. Ephemeral nonces (192 bits) and ephemeral keys are generated afresh for every session. The nonce-collision probability is 2^{-192}.

Harvest-now-decrypt-later attack. If the classical $dh_1, \ldots, dh_4$ terms are later recovered by a quantum adversary, the hybrid key still depends on ss_{kem}, whose protection is tied to ML-KEM-768 and the Module-LWE assumption.

Long-term server-key exposure. Compromise of sk_S permits impersonation of the server in future sessions and makes the static-server DH terms computable from a recorded transcript, including dh_1 and dh_2. The narrower post-session claim is that sk_S alone does not reconstruct a completed hybrid session: it does not reveal the client static secret, the server ephemeral scalar, the ephemeral–ephemeral term dh_4, or the ML-KEM shared secret ss_{kem}, which depends on the destroyed sk_{kem}. The protection statement is therefore scoped to recovery of the recorded hybrid session key, not to impersonation after the server key has been lost.

Timing side channels. MAC comparisons are performed by constant-time bytewise comparison without early exit. The implementation uses constant-time-oriented Ristretto255 and ML-KEM components, but the symbolic models do not analyze microarchitectural leakage.

5.3. Formal Verification

5.3.1. Verification Overview

The protocol was analyzed with two symbolic tools in the Dolev–Yao model: ProVerif 2.05 and Tamarin Prover 1.10.0. The purpose of this analysis is to check the structure of the protocol argument, not to certify the reference implementation line by line. ProVerif is used in separate secrecy and authentication models, whereas Tamarin is applied to a verified surrogate 4DH model. The results are therefore read together with the modeling

boundaries and with the executable artifact that exercises serialization, state handling, and external-call behavior [32, 33].

Table 7: Summary of symbolic verification results and implementation checks

#	Property	ProVerif	Tamarin (surrogate 4DH model)	Rust tests
P1	Session-key secrecy	**confirmed**	**confirmed** (27 steps)	4 tests
P2	Password secrecy on the wire	**confirmed**	**confirmed** (3 steps)	6 tests
P3	Classical forward secrecy	—	**confirmed** (119 steps)	3 tests
P4	Post-quantum forward secrecy	—	argued via P6 within the model	3 tests
P5a	Initiator authentication	**confirmed**	**confirmed** (32 steps)	8 tests
P5b	Responder authentication	**confirmed**	**confirmed** (17 steps)	—
P5c	Injective mutual authentication	**confirmed**	follows from P5a/P5b	—
P6	AND interpretation of hybrid security	—	**supported in the surrogate model** (29 steps)	4 tests
P7	Resistance to offline guessing	—	**supported for the database model without compromise of** σ_{oprf} (4 steps)	6 tests
—	Successful completion	—	**confirmed** (16 steps)	5 tests
Total		**5/5 queries**	**8/8 lemmas**	**157 passed, 1 ignored**

Table 7 combines symbolic and implementation-level checks to show which claims are supported at which layer. ProVerif is most useful for message logic and correspondence properties in a simplified unbounded-session setting, Tamarin for stateful properties such as forward secrecy and the combiner interpretation, and Rust tests for implementation-specific failure modes not represented in the symbolic model.

5.3.2. ProVerif 2.05

Toolchain: ProVerif 2.05 (OCaml 5.4.0), macOS Darwin 25.2.0. Two complementary models were used: a baseline secrecy model and a separate authentication model, both archived in the accompanying artifact package. The split into two models is deliberate: a single saturated model did not yield stable convergence for all queries simultaneously. The ProVerif results provide two compatible but distinct views of the same protocol.

The most informative query for mutual authentication is P5c (injective mutual correspondence): it states that each successful completion on the responder side corresponds to a distinct matching completion on the initiator side for the same pair of parties and the same session key. A positive result means that successful completion on the responder side cannot be explained by replaying a single successful trace across multiple client completions. This result is not a computational proof of the implementation: the model omits side channels, variation in error formatting, timing-related leakage, and a range of network-level details that remain the domain of implementation testing.

5.3.3. Tamarin Prover 1.10.0

Toolchain: Tamarin 1.10.0 (Maude 3.4). A surrogate model of the hybrid combiner was verified; the archived 4DH proof log records a total solving time of **53.84 seconds** for all eight lemmas.

Table 8: Tamarin lemmas established for the surrogate 4DH model

Lemma	Steps	Key property
Session-key secrecy (P1)	27	Without corruption events, the adversary cannot derive the session key
Password secrecy (P2)	3	The password never appears in externally observable model events
Forward secrecy (P3)	119	Post-session compromise of long-term keys does not reveal ephemeral secrets
Initiator authenticity (P5a)	32	Successful client completion requires prior responder participation
Responder authenticity (P5b)	17	Successful server completion requires a valid KE3
AND-style combiner interpretation (P6)	29	Without compromise of the ephemeral secrets, the 4DH and KEM components are not recovered jointly
Offline-guessing resistance in the DB model (P7)	4	Database compromise reveals the record but not the OPRF key
Executability (correctness)	16	A successful protocol-completion trace exists

The verified Tamarin model does not use an embedded Diffie–Hellman equational theory for the full 4DH construction. Instead, it replaces 4DH with an abstract secret $\xi_{\text{4dh}} = h(\langle \text{sk}_C, \text{sk}_S, e_C, e_S \rangle)$ and checks which compromise events are required for the adversary to derive the session key. This abstraction makes automated reasoning feasible and is interpreted as surrogate symbolic evidence rather than a formal proof of every concrete Ristretto255 operation.

Lemma P6 is central to the combiner interpretation. It states that from the establishment of the session key and the absence of disclosure of the initiator's and responder's ephemeral secrets, non-derivability of that key for the adversary follows. The machine verification of this lemma completes positively in 29 steps.

The lemma permits compromise of long-term keys but disallows disclosure of ephemeral ones. Even with knowledge of the parties' long-term se-

crets, the adversary cannot recover the hybrid key material without access to the ephemeral components of both the classical and the post-quantum layers. Within this surrogate model, this supports the AND-style interpretation: derivation of the session key requires weakening of both layers. The lemma provides an indirect argument for P4; it does not replace a separate formalization of post-quantum forward secrecy.

5.3.4. Limits of Interpretation

To avoid overstating the formal results, four principal limitations are recorded.

1. **Surrogate representation of 4DH.** The verified Tamarin model abstracts the 4DH component into a single knowledge function. This facilitates the proof but does not replace analysis of the concrete algebraic properties of group operations.
2. **Split ProVerif models.** Secrecy and authentication are studied not in one universal model but in two separate artifacts. Hence the 5/5 outcome should be interpreted as the aggregate of two mutually consistent models rather than as a monolithic proof.
3. **Boundary of offline-resistance claims.** Lemma P7 and the corresponding Rust tests cover database compromise *without* compromise of σ_{oprf}. If σ_{oprf} is exposed, the offline-resistance property requires separate analysis and should not be transferred automatically from the model.
4. **Gap between model and implementation.** Symbolic verification does not cover side channels, compiler behavior, memory-allocation properties, external-API mistakes, or vulnerabilities in external crates. For this reason, the formal results reported here complement tests and benchmarks rather than substituting for them.

6. Implementation and Experimental Evaluation

6.1. Architecture of the Reference Implementation

The protocol is implemented in **Rust 1.93.1** as a multi-crate workspace. Rust was selected for compile-time memory safety, the ability to keep the cryptographic core free of `unsafe` blocks, and the availability of mature secret-zeroization mechanisms.

Table 9: Source-code metrics of the workspace

Component	Role	LoC	Principal dependencies
Cryptographic core	Primitives, key derivation, and protocol serialization	1513	Ristretto255, ML-KEM, Argon2id
Client pipeline	Initiator logic and client-side protocol phases	900	cryptographic core
Server pipeline	Responder logic and session completion	907	cryptographic core
Interlanguage layer	External API adapter and protocol invocation	2026	client and server pipelines
Total (main code)		**5346**	
Tests		4116	property-based and regression scenarios
Benchmarks		1460	Criterion
Formal models		1806	Tamarin, ProVerif

6.2. Specification-to-Implementation Traceability

The implementation is organized so that the main protocol statements in the paper can be checked against concrete artifacts rather than against informal intent. This traceability is not a proof of correctness, but it reduces the risk that the specification, implementation, benchmarks, and adapter documentation drift into describing different protocols. Table 10 summarizes the correspondence used in the artifact.

Table 10: Traceability between the paper specification and executable artifact

Specification element	Implementation locus	Artifact check
Wire-format lengths and version prefix	Core type constants and protocol serialization/parsing routines	Unit tests reject short, long, and unsupported-version messages; public length accessors expose the same 1273/1377/65-byte authentication sizes.
Account-bound OPRF derivation	OPRF and registration/envelope modules shared by the agent and relay crates	OPRF, envelope, and registration tests cover account-specific derivation, validation failures, and rejection of malformed public inputs.
Hybrid 4DH + ML-KEM key path	Agent authentication, relay authentication, and ML-KEM helper module	Integration and property-oriented tests exercise successful completion, tampered KE2/KE3 rejection, replay rejection, and independent session-key agreement.
Transcript binding of PQ material and acct	Authentication-state construction on both endpoints	Negative tests mutate transcript-bound fields and require authentication failure rather than silent divergence.
State lifetime and zeroization boundary	Agent state, relay state, and external-adapter lifecycle wrappers	Regression tests check state consumption, rejection after invalid transitions, and safe release paths across the C ABI boundary.
Public external error contract	Agent and relay adapter modules plus the public adapter documentation	Adapter lifecycle tests verify that unsupported protocol versions remain distinguishable at the ABI boundary instead of being collapsed into generic authentication failure.

The important point is methodological. The symbolic models describe the message structure and compromise assumptions; the Rust tests check whether the executable state machine follows the same message sizes, labels, version handling, and rejection behavior; and the adapter documentation exposes the subset of those contracts that external callers must obey. This separation makes the artifact more directly auditable because cryptographic claims, implementation invariants, and ABI obligations are related but not treated as interchangeable evidence.

6.3. Memory Safety and Secret Zeroization

For a cryptographic system, security depends not only on the selected primitives but also on the discipline with which secrets are handled in process memory. In the implementation studied here, the cryptographic core and both protocol pipelines are compiled under a strict no-`unsafe`-code policy, whereas operations that require low-level pointer manipulation are confined to a narrow external-call boundary. Such architectural separation does not replace formal analysis of the implementation, but it limits the defect surface at the memory-management boundary: use-after-free defects, double release, loss of correct buffer ownership, and violations of buffer-length invariants.

Control over secret lifetime is equally important. The implementation systematically applies automatic zeroization when objects leave scope together with dedicated wrappers for transient values, so that sensitive intermediate quantities remain in memory only for the minimal interval required. This does not eliminate every possible leak, but it reduces the post-compromise residue of sensitive data in memory dumps, crash logs, and reused buffers. In particular, the following values are zeroized:

- the user password and the OPRF blinding scalar after OPRF finalization;
- the ephemeral ML-KEM secret after decapsulation;
- the classical 4DH material, the transcript hash, and the PRK after derivation of descendant keys;
- the context hash of `acct` after completion of KE2/KE3;
- the initiator and responder state structures upon interruption or any rejected completion path.

6.4. Testing and Reproducibility

The evaluated revision of the workspace contains 157 passing tests and 1 deliberately ignored slow regression scenario. The package includes unit, integration, property-based, and regression tests for the protocol lifecycle, wire format, external API contract, OPRF/KEM correctness, and secure rejection of invalid states.

These checks complement the symbolic analysis rather than duplicate it. Rust tests exercise serialization, version handling, state-lifecycle invariants, and input-boundary conditions in the executable implementation; Criterion 0.5.1 benchmarks [34] quantify the cost of the main primitives and protocol phases; archived ProVerif and Tamarin models document the formal abstractions used for the security claims.

Benchmarking was performed on an Apple M1 Pro platform (8 performance + 2 efficiency cores, 16 GB LPDDR5, macOS 26.2, Rust 1.93.1) in deterministic single-threaded optimized-build mode. Criterion applies bootstrap estimation of the mean and the 95% confidence interval; the values reported below should be interpreted as reproducible estimates for this platform rather than as universal constants for every environment. Reproducing the full test suite, the benchmark suite, and the formal models gives an independent check on the main quantitative and structural claims reported here.

6.5. Benchmarks of Cryptographic Primitives

Table 11: Benchmark results for cryptographic primitives

Primitive	Operation	$\hat{\mu}$	95% CI
Ristretto255	Key-pair generation	16.74 μs	[16.03; 17.84]
Ristretto255	Single DH	46.41 μs	[45.94; 46.98]
Ristretto255	3DH (baseline)	119.04 μs	[117.10; 122.14]
Ristretto255	**4DH (protocol)**	**$\approx$ 164 μs**	[$\approx$162; $\approx$167]
ML-KEM-768	Key-pair generation	41.05 μs	[40.69; 41.68]
ML-KEM-768	Encapsulation	33.62 μs	[32.89; 34.95]
ML-KEM-768	Decapsulation	38.55 μs	[37.65; 40.11]
ML-KEM-768	**Full round**	**122.01 μs**	[119.32; 126.96]
OPRF	Blinding	56.16 μs	[54.37; 59.36]
OPRF	Server evaluation	47.59 μs	[44.97; 52.45]
OPRF	Finalization	65.68 μs	[64.17; 67.89]
HKDF	Extract	1.32 μs	[1.312; 1.319]
HKDF	Expand (64 bytes)	1.21 μs	[1.181; 1.245]
HMAC-SHA-512	256 bytes	1.61 μs	[1.596; 1.641]
XSalsa20-Poly1305	Encrypt 96 bytes	440.79 ns	[430.20; 459.46]
XSalsa20-Poly1305	Decrypt 96 bytes	613.42 ns	[594.15; 636.12]
Argon2id	**MODERATE**	**510.86 ms**	[490.31; 533.80]
Hybrid combiner	HKDF-Extract (160 bytes)	1.48 μs	[1.453; 1.508]

The micro-benchmarks show three stable relationships. First, the full ML-KEM-768 round (122.01 μs) and 4DH ($\approx$ 164 μs) lie in the same order of magnitude, so the post-quantum layer does not alter the computational class of the protocol. Second, Argon2id (510.86 ms in isolated measurement) dominates all other components and determines the perceived authentication latency. Third, the hybrid key-extraction procedure (1.48 μs) is negligible relative to the KSF and does not constitute a separate bottleneck.

6.6. Protocol-Level Benchmarks

Table 12: Benchmark results for protocol-level scenarios

Phase	Measurement scenario	$\hat{\mu}$	95% CI	Dominant component
Registration	Registration-request creation	72.46 μs	[72.15; 72.77]	OPRF blinding
Registration	Server-response creation	45.92 μs	[45.72; 46.16]	OPRF evaluation
Registration	Registration completion	**479.58 ms**	[472.11; 487.75]	**Argon2id**
Authentication	KE1 generation	118.83 μs	[116.59; 121.76]	OPRF + ML-KEM key generation
Authentication	KE2 generation	297.03 μs	[294.81; 299.69]	4DH + ML-KEM encapsulation
Authentication	KE3 generation at the initiator	**492.34 ms**	[478.95; 509.96]	**Argon2id** + ML-KEM decapsulation
Authentication	Final responder verification	1.24 μs	[1.174; 1.323]	MAC verification
Full	End-to-end authentication cycle	**481.43 ms**	[471.73; 495.25]	**Argon2id**

The values in Table 12 are not additive: isolated KE3 generation and the complete authentication scenario were measured in separate benchmark runs with different state-initialization configurations. Consequently, the mean for the full cycle may be slightly lower than the mean for the isolated phase, while the 95% confidence intervals overlap substantially. These measurements should be interpreted in terms of dominant components rather than direct phase-by-phase summation: server-side KE2 generation remains a submillisecond operation, whereas Argon2id contributes the overwhelming share of time.

6.7. Key Quantitative Findings and Implementation Implications

The isolated server profile shows that the responder's cryptographic core remains compatible with high-throughput deployment: isolated KE2 generation takes 254.26 μs, whereas final responder verification takes only 760.82 ns.

This yields a theoretical upper bound of approximately 3.92×10^3 authentications per second per core for a cryptography-bound server, excluding database access, telemetry, and business logic.

For the complete AKE without the KSF, the post-quantum component adds 159.3 μs (+30.1%) of CPU time; against the Argon2id background, however, this excess shrinks to approximately 0.03% of a typical end-to-end scenario. The principal trade-off is not CPU cost but network volume: the complete handshake occupies 2715 bytes in wire format, that is, 2272 bytes more than the classical variant at the same three-message interaction profile.

Comparative claim relative to classical OPAQUE.. Let $M_{\text{opq}}, R_{\text{opq}}, C_{\text{opq}}, W_{\text{opq}}$ denote, respectively, the message count, network round-trip cost, cryptographic computation, and communication volume of canonical 3DH-OPAQUE without the KSF, and let $M_{\text{hyb}}, R_{\text{hyb}}, C_{\text{hyb}}, W_{\text{hyb}}$ denote the corresponding quantities for Hybrid PQ-OPAQUE. Let C_{misc} collect the fixed transcript-binding, serialization, and key-derivation work introduced by the hybrid path. Then

$$\begin{aligned} M_{\text{hyb}} &= M_{\text{opq}} = 3, \\ R_{\text{hyb}} &= R_{\text{opq}} = \tfrac{3}{2}\,\text{RTT}, \\ C_{\text{hyb}} &= C_{\text{opq}} + C_{\text{DH}} + C_{\text{KEM}} + C_{\text{misc}}, \\ W_{\text{hyb}} &= W_{\text{opq}} + \Delta W, \qquad \Delta W = 2272 \text{ bytes}. \end{aligned}$$

On the benchmark platform, this corresponds to a measured increase of $C_{\text{DH}} + C_{\text{KEM}} + C_{\text{misc}} \approx 159.3$ μs without the KSF. Compared with the established baseline of classical OPAQUE, the cost for honest parties rises additively, and the interaction profile is unchanged. Within the present threat model, explicit compromise of the final key material is not reducible to weakening the classical contribution alone; it is interpreted as requiring compromise of both the classical and post-quantum layers.

7. Discussion of Results

7.1. Interpretation of the Formal and Experimental Evidence

The evidence should be read as layered rather than monolithic. ProVerif addresses secrecy and correspondence in symbolic models, Tamarin records stateful properties of the surrogate 4DH construction, and the Rust test suite checks executable behavior that symbolic tools do not model: serialization,

Table 13: Comparison of Hybrid PQ-OPAQUE with canonical 3DH-OPAQUE

Aspect	3DH-OPAQUE	Hybrid PQ-OPAQUE	Interpretation
Interaction profile	3 messages, 1.5 RTTs	3 messages, 1.5 RTTs	Protocol interaction is unchanged
Computational complexity for honest parties	Baseline 3DH path	+1 additional DH, + KEM, + fixed transcript/KDF work	The increase is additive rather than structural
Communication complexity	Baseline handshake volume	+2272 bytes in wire format	The principal practical trade-off shifts to transport
Key-compromise condition	Weakening of the classical contribution	Compromise of both the classical and PQ layers	The recovery condition is stricter within the present threat model

state lifetime, boundary checks, version handling, and the external API contract. These layers point to the same protocol description, but they do not have the same evidentiary status.

The least automatic part of the argument is the AND-style reading of the hybrid combiner. It is supported by the extractor rationale and by the surrogate Tamarin lemma, but it is still weaker than a full computational proof. This distinction is important because the paper is not claiming that the implementation is machine-verified; it claims that the design, models, tests, and measurements are mutually consistent under the stated assumptions.

Table 14: Scope of the main claims and supporting evidence

Claim	Evidence used in the paper	Boundary of the claim
Fixed-round hybridization	Explicit message syntax, transcript definitions, wire-format accounting, and Rust serialization tests	The construction is OPAQUE-like and does not claim complete RFC 9807 conformance.
Transcript-bound post-quantum contribution	Key-schedule equations, context binding through acct, formal models, and integration tests	The formal tools reason about symbolic abstractions, not the compiled binary or side-channel behavior.
Resistance to database-only offline guessing	OPRF-key separation, database-compromise model, negative regression tests, and the stated trust boundary for σ_{oprf}	The claim does not survive compromise of the OPRF root secret or a deployment that stores it with the password database.
AND-style hybrid key dependence	HKDF extractor rationale, the surrogate 4DH Tamarin lemma, and agreement between the specification and implementation paths	This is evidence for the intended combiner interpretation, not a full computational proof of the hybrid PAKE.
Practical overhead is dominated by the KSF	Criterion measurements for primitive costs, end-to-end authentication, and message-size accounting	The absolute timings are platform-specific and must be recalibrated for mobile, embedded, or high-latency deployments.

7.2. Performance, KSF Parameters, and the Trust Boundary

The performance profile is governed mainly by Argon2id; ML-KEM-768 and 4DH remain sub-millisecond components. The practical deployment question is therefore not the absolute cost of ML-KEM-768, but the choice of KSF profile for the target client population. The selected profile is appropriate for web and B2B settings, whereas mobile and edge deployments may require separate KSF calibration and stricter rate limiting.

The trust boundary around σ_{oprf} is not a cosmetic detail. Resistance to offline guessing is claimed only for database compromise *without* compromise of that secret. In operational terms, σ_{oprf} functions as the root cryptographic secret of the service and requires separated storage, segregated access, minimal memory residency, and a rotation policy that treats OPRF-key derivation as privileged cryptographic state.

A practical advantage of the migration path is that the post-quantum component remains ephemeral and does not enlarge the registration record. Migration is concentrated in the application-layer authentication logic, version negotiation, and contextual binding of acct. The growth of the wire format to 2715 bytes, however, makes transport constraints more consequential than in classical OPAQUE: for TCP/QUIC this is largely a buffering issue, whereas datagram deployments also face fragmentation and MTU concerns.

7.3. Migration, Compatibility, and Operational Viability

The construction can be deployed without redesign of the entire identity stack for two reasons. First, the registration record does not accumulate long-lived post-quantum material, because ML-KEM appears only in the ephemeral authentication path. Second, explicit wire-format versioning and contextual binding of acct permit staged rollout without invalidating existing user state. For operators, migration is therefore closer to controlled protocol evolution than to complete credential re-issuance.

Operational compatibility also matters. Authentication interacts with login-attempt limits, logging, account recovery, anti-fraud analysis, and audit requirements. The implementation therefore emphasizes deterministic serialization, explicit version checking, and stable external-call boundaries. Failure modes should also remain distinguishable: cryptographic failure, version-negotiation failure, and transport failure have different operational meanings, and collapsing them into a generic failed login would obscure diagnosis during deployment. Comparable deployment constraints arise in industrial mon-

itoring and inspection platforms that combine UAV operation, multimodal imagery, and SCADA-linked telemetry [35, 36, 37].

7.4. Reproducibility and Artifact Transparency

The artifact is useful because it forces the specification to meet the implementation boundary. The same wire sizes, transcript fields, key schedule, and error paths appear in the paper, the Rust crates, and the adapter documentation. This makes it possible to compare the written protocol against executable behavior rather than treating the paper and the implementation as separate claims.

This linkage also keeps the interpretation of the evidence disciplined. The AND-style argument belongs to the Tamarin abstraction and the combiner rationale. Parsing correctness, rejection of non-canonical points, version handling, and prevention of state reuse are properties of the reference implementation and its tests. For post-quantum migration work, that separation is useful: it makes the strengths of the artifact visible without hiding the parts that still require stronger analysis.

7.5. Validity, Limitations, and Future Directions

The work has several material limitations. The implementation is OPAQUE-like rather than fully conformant to RFC 9807 and the associated standard interfaces [5]. The symbolic models do not replace a computational proof of the implementation code, and the benchmark measurements were obtained on a single hardware platform; consequently, absolute microsecond-level values should be transferred to other architectures with caution. Finally, ML-KEM-768 is a relatively recent standard, so hybridization reduces dependence on a single assumption but does not remove the need for continued cryptanalysis of the post-quantum layer.

Further development of the work is most naturally directed along four lines:

1. adaptation to RFC 9807 and the associated OPAQUE/OPRF ecosystem;
2. formalization of the hybrid extension in a computational or UC setting [25];
3. investigation of alternative KSF profiles and ML-KEM variants for mobile and IoT scenarios;

4. integration of external post-quantum signatures for settings in which non-repudiation is required, without altering the baseline HMAC-based handshake confirmation.

8. Conclusion

The results indicate that OPAQUE can be hybridized with ML-KEM-768 without increasing the number of protocol messages. The construction fixes the wire format, extends the transcript with post-quantum and account-binding material, and derives key material from a hybrid combiner over 4DH and ML-KEM-768. Within the adopted symbolic models, the resulting key schedule makes final-key recovery depend on both the classical and post-quantum contributions.

The evidence is deliberately mixed. ProVerif confirms secrecy and authentication in separate models, Tamarin establishes 8/8 lemmas in the archived surrogate 4DH model, and the reference implementation exercises the wire format, state machine, tests, and benchmark paths. On the benchmark platform, end-to-end authentication takes 481.43 ms, Argon2id is the main latency component, and the additional cryptographic cost of the post-quantum layer is small in processor time but substantial in message expansion.

The limits of the result are part of the claim. The implementation is not a full RFC 9807 instantiation, the symbolic models do not constitute a computational proof of the code, and the dictionary-resistance claim applies only to compromise of the server database without compromise of σ_{oprf}. Within those boundaries, the work gives a reproducible starting point for a post-quantum augmented-PAKE profile that can be studied, measured, and compared against independent implementations.

Artifact Availability

The implementation and verification artifact is available as a public source artifact:

https://github.com/oleksandrmelnychenko/aura-security-opaque-rs

The Rust workspace contains the protocol crates, deterministic and property-oriented tests, external API bindings, and benchmark harnesses. The formal directory contains the ProVerif and Tamarin models together with archived verification logs. For the artifact state described in this paper, the Rust

workspace test run reports 157 passed tests and one intentionally ignored slow regression, while the archived symbolic evidence records 5/5 ProVerif queries and 8/8 Tamarin lemmas for the documented models.

The artifact is intended to make the boundary between specification and implementation inspectable. Message sizes, transcript fields, labels, version checks, and error conventions are represented in both the paper and the executable code. The artifact also retains supporting documentation for the external API layer and reproducibility workflow, so that future revisions can be checked against the same implementation-level contracts rather than against the prose alone.

Acknowledgment

OpenAI Codex was used during manuscript preparation to assist with English-language drafting, stylistic revision, and LaTeX editing in portions of the abstract, introduction, discussion, conclusion, and related connective prose. The system was not used as an authoritative source for technical claims. All protocol descriptions, equations, references, tables, implementation details, and experimental results were independently reviewed, verified, and approved by the author.

References

[1] Bonneau J., Herley C., van Oorschot P. C., Stajano F. The quest to replace passwords: a framework for comparative evaluation of web authentication schemes. In: Proceedings of the IEEE Symposium on Security and Privacy, 2012, pp. 553–567.

[2] Florencio D., Herley C. A large-scale study of web password habits. In: Proceedings of the International Conference on World Wide Web, 2007, pp. 657–666.

[3] Bellare M., Pointcheval D., Rogaway P. Authenticated key exchange secure against dictionary attacks. In: EUROCRYPT 2000, LNCS 1807, Springer, 2000, pp. 139–155.

[4] Jarecki S., Krawczyk H., Xu J. OPAQUE: an asymmetric PAKE protocol secure against pre-computation attacks. In: EUROCRYPT 2018, LNCS 10822, Springer, 2018, pp. 456–486.

[5] Bourdrez D., Krawczyk H., Lewi K., Wood C. A. The OPAQUE Augmented Password-Authenticated Key Exchange (aPAKE) Protocol. IRTF RFC 9807, July 2025. URL: https://www.rfc-editor.org/info/rfc9807.

[6] Shor P. W. Algorithms for quantum computation: discrete logarithms and factoring. In: Proceedings of the IEEE Symposium on Foundations of Computer Science, 1994, pp. 124–134.

[7] Bernstein D. J., Lange T. Post-quantum cryptography. Nature 549(7671) (2017) 188–194.

[8] Roetteler M., Naehrig M., Svore K. M., Lauter K. Quantum resource estimates for computing elliptic curve discrete logarithms. In: ASIACRYPT 2017, LNCS 10625, Springer, 2017, pp. 241–270.

[9] Babbush R., Zalcman A., Gidney C., Broughton M., Khattar T., Neven H., Bergamaschi T., Drake J., Boneh D. Securing elliptic curve cryptocurrencies against quantum vulnerabilities: resource estimates and mitigations. Google Quantum AI whitepaper, March 30, 2026. URL: https://quantumai.google/static/site-assets/downloads/cryptocurrency-whitepaper.pdf.

[10] Mosca M. Cybersecurity in an era with quantum computers: will we be ready? IEEE Security & Privacy 16(5) (2018) 38–41.

[11] National Institute of Standards and Technology. Module-Lattice-Based Key-Encapsulation Mechanism Standard: FIPS 203. 2024. URL: https://csrc.nist.gov/pubs/fips/203/final.

[12] National Institute of Standards and Technology. Module-Lattice-Based Digital Signature Standard: FIPS 204. 2024. URL: https://csrc.nist.gov/pubs/fips/204/final.

[13] Stebila D., Fluhrer S., Gueron S. Hybrid key exchange in TLS 1.3. IETF Internet-Draft draft-ietf-tls-hybrid-design-16, work in progress, September 2025. URL: https://datatracker.ietf.org/doc/draft-ietf-tls-hybrid-design/.

[14] Brendel J., Fiedler R., Günther F., Jacobsen C., Poettering B. Post-quantum security of the Signal protocol. In: SAC 2020, LNCS 12804, Springer, 2020, pp. 567–591.

[15] Hülsing A., Ning K.-C., Schwabe P., Weber F., Zimmermann P. R. Post-quantum WireGuard. In: Proceedings of the IEEE Symposium on Security and Privacy, 2021, pp. 304–321.

[16] Arriaga A., Barbosa M., Jarecki S., Skrobot M. C'est très CHIC: a compact password-authenticated key exchange from lattice-based KEM. Cryptology ePrint Archive, Paper 2024/308, 2024. URL: https://eprint.iacr.org/2024/308.

[17] Lyu Y., Liu S., Han S. Universal composable password authenticated key exchange for the post-quantum world. In: EUROCRYPT 2024, LNCS 14657, Springer, 2024, pp. 120–150.

[18] Hesse J., Rosenberg M. PAKE combiners and efficient post-quantum instantiations. Cryptology ePrint Archive, Paper 2024/1621, 2024. URL: https://eprint.iacr.org/2024/1621.

[19] Lyu Y., Liu S. Hybrid password authentication key exchange in the UC framework. Cryptology ePrint Archive, Paper 2024/1630, 2024. URL: https://eprint.iacr.org/2024/1630.

[20] Boyko V., MacKenzie P., Patel S. Provably secure password-authenticated key exchange using Diffie–Hellman. In: EUROCRYPT 2000, LNCS 1807, Springer, 2000, pp. 156–171.

[21] Bellovin S. M., Merritt M. Encrypted key exchange: password-based protocols secure against dictionary attacks. In: Proceedings of the IEEE Symposium on Security and Privacy, 1992, pp. 72–84.

[22] Ladd W. SPAKE2, a password-authenticated key exchange. IRTF RFC 9382, September 2023. URL: https://www.rfc-editor.org/info/rfc9382.

[23] Haase B., Labrique B. AuCPace: efficient verifier-based PAKE protocol tailored for the IIoT. IACR Transactions on Cryptographic Hardware and Embedded Systems 2019(2) (2019) 1–48.

[24] Wu T. The Secure Remote Password protocol. In: Proceedings of NDSS, 1998, pp. 97–111.

[25] Canetti R. Universally composable security: a new paradigm for cryptographic protocols. In: Proceedings of the IEEE Symposium on Foundations of Computer Science, 2001, pp. 136–145.

[26] de Valence H., Grigg J., Hamburg M., Lovecruft I., Tankersley G., Valsorda F. The ristretto255 and decaf448 groups. IRTF RFC 9496, December 2023. URL: https://www.rfc-editor.org/info/rfc9496.

[27] Esgin M. F., Steinfeld R., Tairi E., Xu J. LeOPaRd: towards practical post-quantum oblivious PRFs via interactive lattice problems. Cryptology ePrint Archive, Paper 2024/1615, 2024. URL: https://eprint.iacr.org/2024/1615.

[28] Castryck W., Decru T. An efficient key recovery attack on SIDH. In: EUROCRYPT 2023, LNCS 14008, Springer, 2023, pp. 423–447.

[29] Vos J., Jarecki S., Wood C. A. Hybrid Post-Quantum Password Authenticated Key Exchange. Individual Internet-Draft draft-vos-cfrg-pqpake-00, April 2025. URL: https://datatracker.ietf.org/doc/draft-vos-cfrg-pqpake/00/.

[30] Biryukov A., Dinu D., Khovratovich D. Argon2: new generation of memory-hard functions for password hashing. In: Proceedings of IEEE European Symposium on Security and Privacy, 2016, pp. 292–302.

[31] Krawczyk H., Eronen P. HMAC-based Extract-and-Expand Key Derivation Function (HKDF). IETF RFC 5869, 2010.

[32] Cremers C., Mauw S. Operational Semantics and Verification of Security Protocols. Springer, 2012.

[33] Basin D., Cremers C., Meadows C. Model checking security protocols. In: Handbook of Model Checking. Springer, 2018, pp. 727–762.

[34] Heisler B. Criterion.rs: statistics-driven micro-benchmarking in Rust. Project site. URL: https://github.com/bheisler/criterion.rs.

[35] Svystun S., Scisło L., Pawlik M., Melnychenko O., Radiuk P., Savenko O., Sachenko A. DyTAM: accelerating wind turbine inspections with dynamic UAV trajectory adaptation. Energies 18(7) (2025) 1823. DOI: https://doi.org/10.3390/en18071823.

[36] Svystun S., Melnychenko O., Radiuk P., Savenko O., Sachenko A., Lysyi A. Thermal and RGB images work better together in wind turbine damage detection. International Journal of Computing 23(4) (2024) 526–535. DOI: https://doi.org/10.47839/ijc.23.4.3752.

[37] Lysyi A., Sachenko A., Radiuk P., Lysyi M., Melnychenko O., Ishchuk O., Savenko O. Enhanced fire hazard detection in solar power plants: an integrated UAV, AI, and SCADA-based approach. Radioelectronic and Computer Systems (2) (2025) 99–117. DOI: https://doi.org/10.32620/reks.2025.2.06.